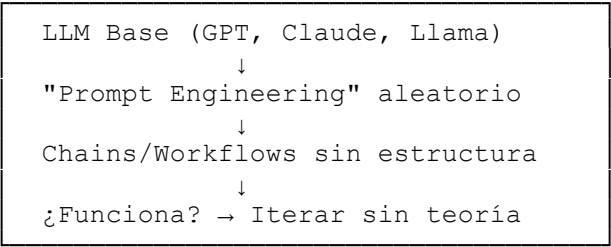


ACT-R como Marco Meta-Cognitivo para Sistemas LLM

🔍 El Problema Actual (Octubre 2025)

Estado de la Industria de LLMs

✖ ENFOQUE ACTUAL (Ad-hoc)



Problemas:

- No hay teoría unificadora
- Cada equipo reinventa la rueda
- No hay principios de diseño
- Difícil predecir comportamiento
- Imposible transferir conocimiento entre dominios

Lo que Está Faltando

Aspecto	Estado Actual	Lo que Falta
Workflows	Ad-hoc, trial & error	Principios de diseño cognitivo
Prompts	"Arte", no ciencia	Teoría de representación del conocimiento
Chains	Secuencias arbitrarias	Modelo de control ejecutivo
Context Management	Heurísticas	Teoría de memoria de trabajo
Multi-step Reasoning	ReACT, CoT, pero ¿por qué?	Modelo de resolución de problemas
Error Handling	Reintentos ciegos	Modelo de detección y corrección de errores

✓ PROPUESTA: ACT-R como Meta-Modelo

La Visión Correcta

✓ ENFOQUE PROPUESTO (Fundamentado)

```
ACT-R (Arquitectura Cognitiva)
• Teoría de memoria
• Teoría de atención
• Teoría de aprendizaje
• Teoría de control ejecutivo
    ↓
PRINCIPIOS DE DISEÑO
(derivados de ciencia cognitiva)
    ↓
Meta-Modelo de Workflows LLM
    ↓
Implementación con LLMs
(prompts, chains, tools)
```

Ventajas:

- ✓ Teoría unificadora basada en ciencia
- ✓ Principios transferibles entre dominios
- ✓ Comportamiento predecible
- ✓ Explicabilidad inherente
- ✓ Optimización basada en evidencia



ACT-R como Framework Meta-Cognitivo

1. Gestión de Memoria → Context Management

Problema actual con LLMs:

```
# Típico problema: Context window lleno
context = load_all_documents() # ✗ 100K tokens
response = llm.generate(context + query) # Lento, costoso, ineficiente
```

Solución inspirada en ACT-R:

```
class ACTRContextManager:
    """
    Gestión de contexto basada en principios de memoria declarativa
    """
    def __init__(self):
        self.context_chunks = [] # Como chunks en memoria
        self.activation_tracker = {} # Activación de cada chunk

    def add_context(self, chunk, base_activation=1.0):
        """Añade contexto con activación inicial"""
        self.context_chunks.append({
            'content': chunk,
            'activation': base_activation,
            'last_access': time.now(),
            'access_count': 0
        })

    def retrieve_relevant_context(self, query, max_tokens=4000):
        """
        Recupera SOLO el contexto relevante
        Inspirado en: ACT-R activation-based retrieval
        """
        # Calcular activación para cada chunk
        for chunk in self.context_chunks:
            # Base-level activation (frecuencia + recencia)
            recency_factor = self._calculate_decay(chunk['last_access'])
            frequency_factor = math.log(chunk['access_count'] + 1)

            # Spreading activation (similitud con query)
            relevance = self._semantic_similarity(query,
            chunk['content'])

            chunk['activation'] = (
                chunk['activation'] * recency_factor +
                frequency_factor +
                relevance * 0.5 # Spreading weight
            )

        # Recuperar chunks sobre threshold
        active_chunks = [
            c for c in self.context_chunks
            if c['activation'] > self.threshold
```

```

    ]

    # Ordenar por activación y limitar tokens
    active_chunks.sort(key=lambda x: -x['activation'])

    selected_context = []
    total_tokens = 0

    for chunk in active_chunks:
        chunk_tokens = count_tokens(chunk['content'])
        if total_tokens + chunk_tokens <= max_tokens:
            selected_context.append(chunk['content'])
            total_tokens += chunk_tokens
            chunk['access_count'] += 1
            chunk['last_access'] = time.now()

    return "\n".join(selected_context)

def _calculate_decay(self, last_access):
    """ACT-R decay function:  $t^{-d}$ """
    time_since_access = time.now() - last_access
    return time_since_access ** -0.5 # d = 0.5 típico

def _semantic_similarity(self, query, content):
    """Spreading activation basado en similitud semántica"""
    # Usar embeddings del LLM
    return cosine_similarity(
        embed(query),
        embed(content)
    )

# USO
context_mgr = ACTRContextManager()

# Añadir documentos con prioridades
context_mgr.add_context(important_doc, base_activation=2.0)
context_mgr.add_context(secondary_doc, base_activation=1.0)

# Recuperar solo lo relevante (no todo el contexto)
relevant = context_mgr.retrieve_relevant_context(user_query)
response = llm.generate(relevant + user_query)

```

Beneficios:

-  **Eficiencia:** Solo tokens relevantes
-  **Escalabilidad:** Maneja corpus grandes
-  **Cognitivamente plausible:** Simula memoria humana
-  **Adaptativo:** Aprende qué es importante

2. Control Ejecutivo → Workflow Orchestration

Problema actual:

```
# Workflow típico: Secuencia fija sin adaptación
def medical_diagnosis_workflow(symptoms):
    # ✗ Siempre hace todos los pasos, incluso si no es necesario
    step1 = llm.generate("Analiza síntomas: " + symptoms)
    step2 = llm.generate("Busca diagnósticos posibles: " + step1)
    step3 = llm.generate("Verifica diagnóstico: " + step2)
    step4 = llm.generate("Recomienda tratamiento: " + step3)
    return step4
```

Solución inspirada en ACT-R:

```
class ACTRWorkflowOrchestrator:
    """
    Orquestación de workflow basada en goal-directed behavior de ACT-R
    """
    def __init__(self, llm):
        self.llm = llm
        self.goal_stack = [] # Pila de objetivos (como ACT-R)
        self.working_memory = {} # Estado actual
        self productions = [] # Reglas de producción

    def add_production(self, name, condition, action):
        """Añade regla de producción"""
        self productions.append({
            'name': name,
            'condition': condition,
            'action': action,
            'utility': 1.0
        })

    def execute_workflow(self, initial_goal):
        """
        Ejecuta workflow adaptativamente
        Inspirado en: ACT-R production system + goal management
        """
        self.goal_stack = [initial_goal]
        trace = []

        while self.goal_stack:
            current_goal = self.goal_stack[-1]

            # FASE 1: Elaboration (inferir sobre estado actual)
            self._elaborate()

            # FASE 2: Seleccionar producción aplicable
            applicable = self._find_applicable productions(current_goal)

            if not applicable:
                # IMPASSE: No hay regla aplicable
                self._handle_impasse(current_goal)
                continue
```

```

# Conflict resolution (utility-based)
selected = max(applicable, key=lambda p: p['utility'])

# FASE 3: Ejecutar producción
trace.append({
    'goal': current_goal,
    'production': selected['name'],
    'state': dict(self.working_memory)
})

result = selected['action'](self.working_memory, self.llm)

# Actualizar estado
self._update_working_memory(result)

# FASE 4: Goal management
if result.get('goal_achieved'):
    self.goal_stack.pop()
elif result.get('subgoal'):
    self.goal_stack.append(result['subgoal'])

return trace

def _elaborate(self):
    """
    Fase de elaboración: inferir información adicional
    SIN llamar al LLM (solo razonamiento local)
    """
    # Ejemplo: Si tenemos síntomas + diagnóstico → inferir urgencia
    if 'symptoms' in self.working_memory and 'diagnosis' in
self.working_memory:
        critical_symptoms = ['chest_pain', 'difficulty_breathing']
        has_critical = any(s in self.working_memory['symptoms']
                           for s in critical_symptoms)
        self.working_memory['urgency'] = 'high' if has_critical else
'normal'

def _handle_impasse(self, goal):
    """
    Manejo de impasses (como ACT-R)
    Cuando no hay regla aplicable → crear subgoal
    """
    if goal['type'] == 'diagnose':
        # Si no podemos diagnosticar → necesitamos más información
        self.goal_stack.append({
            'type': 'gather_information',
            'target': 'additional_symptoms'
        })
    else:
        # No podemos resolver → reportar
        self.working_memory['error'] = f"Cannot achieve goal: {goal}"
        self.goal_stack.pop()

# DEFINIR REGLAS (Meta-modelo del workflow)
orchestrator = ACTRWorkflowOrchestrator(llm)

```

```

# Regla 1: Si hay síntomas pero no diagnóstico → diagnosticar
orchestrator.add_production(
    name="diagnose-from-symptoms",
    condition=lambda wm: 'symptoms' in wm and 'diagnosis' not in wm,
    action=lambda wm, llm: {
        'diagnosis': llm.generate(f"Diagnose: {wm['symptoms']}"),
        'goal_achieved': False
    }
)

# Regla 2: Si hay diagnóstico confirmado → recomendar tratamiento
orchestrator.add_production(
    name="recommend-treatment",
    condition=lambda wm: 'diagnosis' in wm and wm.get('confidence', 0) >
0.7,
    action=lambda wm, llm: {
        'treatment': llm.generate(f"Treat: {wm['diagnosis']}"),
        'goal_achieved': True
    }
)

# Regla 3: Si diagnóstico incierto → buscar más evidencia
orchestrator.add_production(
    name="gather-more-evidence",
    condition=lambda wm: 'diagnosis' in wm and wm.get('confidence', 0) <=
0.7,
    action=lambda wm, llm: {
        'subgoal': {'type': 'gather_information', 'target':
'differential_tests'},
        'goal_achieved': False
    }
)

# EJECUTAR (adaptativamente, no secuencia fija)
trace = orchestrator.execute_workflow({
    'type': 'diagnose',
    'data': {'symptoms': ['fever', 'cough']}
})

```

Beneficios:

-  **Adaptativo:** Workflow cambia según el caso
-  **Eficiente:** Solo pasos necesarios
-  **Explicable:** Trace muestra razonamiento
-  **Robusto:** Maneja impasses
-  **Transferible:** Mismas reglas aplican a diferentes casos

3. Aprendizaje → Optimización de Workflows

Problema actual:

```
# ✗ No hay aprendizaje sistemático
# Cada nueva versión del sistema es un rediseño manual
```

Solución inspirada en ACT-R (Chunking + Utility Learning):

```
class ACTRWorkflowLearner:
    """
    Aprendizaje de workflows basado en chunking y utility learning
    """
    def __init__(self):
        self.production_utilities = {} # Utility de cada regla
        self.chunks = [] # Secuencias aprendidas
        self.success_history = []

    def learn_from_execution(self, trace, outcome):
        """
        Aprende de ejecución exitosa/fallida
        Inspirado en: ACT-R utility learning + chunking
        """
        # 1. UTILITY LEARNING: Ajustar utilidad de producciones
        success = outcome.get('success', False)
        reward = 1.0 if success else -0.5

        for step in trace:
            production_name = step['production']

            if production_name not in self.production_utilities:
                self.production_utilities[production_name] = 1.0

            # Actualización de utilidad (reinforcement learning)
            current_utility = self.production_utilities[production_name]
            learning_rate = 0.2

            self.production_utilities[production_name] = (
                current_utility + learning_rate * (reward -
current_utility)
            )

            # 2. CHUNKING: Compilar secuencias frecuentes
            if success and len(trace) >= 3:
                sequence = tuple(s['production'] for s in trace)

                # ¿Esta secuencia ya ocurrió antes?
                similar_chunks = [c for c in self.chunks if c['sequence'] ==
sequence]

                if similar_chunks:
                    # Incrementar frecuencia
                    similar_chunks[0]['frequency'] += 1
                else:
                    # Crear nuevo chunk
```

```

        self.chunks.append({
            'sequence': sequence,
            'frequency': 1,
            'conditions': trace[0]['state'],
            'outcome': outcome
        })

    # Si un chunk es muy frecuente → crear regla compilada
    for chunk in self.chunks:
        if chunk['frequency'] >= 5: # Threshold
            self._create_compiled_production(chunk)

def _create_compiled_production(self, chunk):
    """
    Crea producción compilada que ejecuta secuencia completa
    Inspirado en: ACT-R chunking (compilar experiencia)
    """
    print(f"📄 Chunk aprendido: {chunk['sequence']}")
    print(f"    Puede ejecutarse como una sola regla optimizada")

    # En implementación real, esta nueva regla:
    # - Se ejecuta más rápido (un solo paso LLM)
    # - Tiene mayor utility (preferida sobre secuencia)
    # - Reduce llamadas al LLM

# USO
learner = ACTRWorkflowLearner()

# Ejecutar múltiples casos
for case in training_cases:
    trace = orchestrator.execute_workflow(case)
    outcome = evaluate(trace, case.expected)
    learner.learn_from_execution(trace, outcome)

# Después de aprendizaje:
print("Utilidades aprendidas:")
for prod, utility in learner.production_utilities.items():
    print(f"    {prod}: {utility:.2f}")

print("\nChunks frecuentes:")
for chunk in sorted(learner.chunks, key=lambda c: -c['frequency']):
    print(f"    {chunk['sequence']}: {chunk['frequency']} veces")

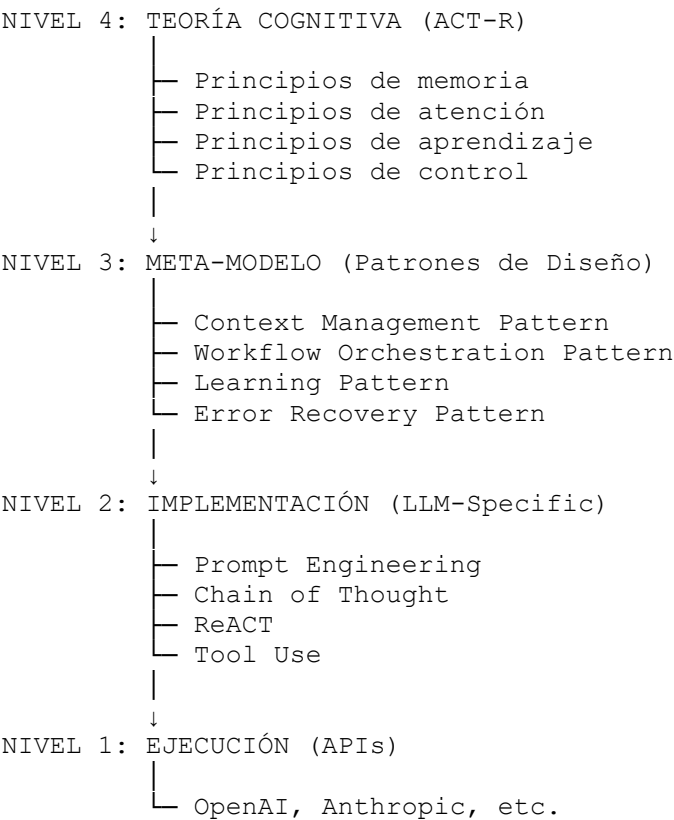
```

Beneficios:

-  **Optimización automática:** Aprende qué funciona
-  **Eficiencia creciente:** Compila secuencias frecuentes
-  **Basado en evidencia:** No adivinanzas
-  **Transferible:** Conocimiento reutilizable

🎓 Meta-Modelado: Framework Completo

Jerarquía de Abstracción



📊 Comparación: Con vs Sin ACT-R

Aspecto	Sin ACT-R (Actual)	Con ACT-R (Propuesto)
Diseño	Trial & error	Principios fundamentados
Escalabilidad	Cada dominio desde cero	Transferencia de patrones
Explicabilidad	"El LLM lo hizo"	Trace cognitivo claro
Optimización	Manual, lento	Aprendizaje automático
Robustez	Frágil a edge cases	Manejo sistemático de impasses
Colaboración	Difícil compartir	Vocabulario común
Investigación	Aneecdótico	Experimental riguroso

🚀 Contribución al Campo

Lo que ACT-R aporta a LLMs:

1. **Teoría Unificadora**
 - No más "arte" del prompt engineering
 - Principios derivables de ciencia cognitiva
 2. **Meta-Modelo Transferible**
 - Patrones aplicables entre dominios
 - Reutilización de conocimiento arquitectónico
 3. **Optimización Sistemática**
 - No más hyperparameter tuning ciego
 - Parámetros con significado cognitivo
 4. **Explicabilidad Inherente**
 - Trace de razonamiento natural
 - Auditable y verificable
 5. **Escalabilidad**
 - Gestión eficiente de contexto
 - Workflows adaptativos
 6. **Base Científica**
 - 30+ años de investigación en cognición
 - Validación experimental posible
-

Visión para tu Maestría

Tesis/Proyectos Potenciales:

1. **"ACT-R como Framework para LLM Workflows"**
 - Formalizar patrones de diseño
 - Validar con casos reales
 2. **"Gestión de Contexto Cognitivamente Inspirada"**
 - Implementar activation-based retrieval
 - Comparar con RAG tradicional
 3. **"Aprendizaje de Workflows via Chunking"**
 - Compilación automática de secuencias
 - Optimización de costos de LLM
 4. **"Meta-Modelado de Sistemas Multi-Agente LLM"**
 - ACT-R para coordinar múltiples LLMs
 - Inspirado en cognición social
-

Contribución Potencial

- Llenar el vacío teórico en ingeniería de LLMs
- Proporcionar framework fundamentado científicamente
- Permitir diseño sistemático (no trial & error)

- Facilitar transferencia de conocimiento entre dominios

Escalabilidad de ACT-R:

- ☒ Aplicable desde tareas simples hasta complejas
 - ☒ Transferible entre dominios (médico, legal, educación)
 - ☒ Integratable con cualquier LLM
 - ☒ Extensible con nuevos módulos cognitivos
-

Lectura Recomendada

Para fundamentar esta visión:

1. **ACT-R Theory:** Anderson (2007) "How Can the Human Mind Occur in the Physical Universe?"
2. **Cognitive Architectures:** Laird et al. (2017) "A Standard Model of the Mind"
3. **LLM + Cognitive Models:** Reciente (2024-2025) - Campo emergente